

chat, preciso fazer 4 exercicios de poo em c++, mas quero tentar fazer sozinho. me ajuda so quando eu travar?

claro kkkkk

faz assim: você tenta cada exercício sozinho e me manda exatamente onde travou — pode ser o enunciado, seu código, erro do compilador ou até “não sei nem como começar”.

eu te ajudo só naquele ponto, sem entregar o exercício inteiro de cara. posso ir dando dica por dica e, se seu código estiver errado, explicar o motivo e ajudar você a corrigir mantendo o código o mais simples possível.

o fazendo um aqui +- assim
void duelar(banda rival)
mas quando diminuo a energia dela, depois na main
continua o mesmo valor. pq?

porque desse jeito você provavelmente está passando a `banda` por valor:

```
</> C++
```

```
void duelar(banda rival)
```

isso cria uma cópia de `rival` dentro da função. aí você diminui a energia da cópia, mas a original que está na `main` continua igual.

pra alterar a original, passa por referência:

```
</> C++
```

```
void duelar(banda &rival)
```

o `&` faz a função trabalhar com o próprio objeto que veio da `main`, não com uma cópia.

exemplo bem simples:

```
</> C++
```

```
void duelar(banda &rival) {  
    rival.energia -= 10;  
}
```

aí depois do `duelar(rival);`, a energia dela na `main` também vai ter diminuído.

```
to tentando fazer rival.energia = rival.energia -  
potencia_som. posso simplificar pra rival.energia -=  
potencia_som?
```

sim, exatamente.

```
</> C++
```

```
rival.energia -= potencia_som;
```

é a forma simplificada de:

```
</> C++
```

```
rival.energia = rival.energia - potencia_som;
```

faz a mesma coisa.

show, deu certo aq, vamos pro ex 2

boa kkkkk manda o exercício 2 e o que você já tentou. vou só te destravando sem entregar tudo de cara.

```
setter em c++? seria tipo void setnome(string n) { nome =  
n; }?  
e getter precisa retornar o mesmo tipo do atributo? tipo  
string getnome() e int getrank()?
```

isso mesmo.

```
</> C++
```

```
void setNome(string n) {  
    nome = n;  
}
```

e o getter normalmente retorna o mesmo tipo do atributo:

```
</> C++
```

```
string getNome() {  
    return nome;  
}
```

```
int getRank() {  
    return rank;  
}
```

então pensa assim: **setter altera**, por isso geralmente é `void` ; **getter devolve**, então o tipo dele tem que bater com o que ele retorna.

se `nome` é `string` , `getter string` . se `rank` é `int` , `getter int` .

```
meu getter ta dando erro porque fiz void getnome() {  
    return nome; }. pq?
```

porque `void` significa que a função **não retorna nada**.

Então isso dá erro:

```
</> C++  
  
void getNome() {  
    return nome;  
}
```

porque você está tentando retornar uma `string` numa função declarada como `void` .

Se `nome` é `string` , faz assim:

```
</> C++  
  
string getNome() {  
    return nome;  
}
```

E a lógica vale pros outros também:

```
</> C++  
  
int getRank() {  
    return rank;  
}
```

O tipo antes do nome da função precisa combinar com o tipo do valor que você retorna.

os getters e setter tem que estar privados ou public?

geralmente, os atributos ficam `private` e os getters/setters ficam `public` .

tipo:

```
</> C++  
  
class banda {  
private:  
    string nome;  
    int rank;  
  
public:  
    void setNome(string n) {  
        nome = n;  
    }  
  
    string getNome() {  
        return nome;  
    }  
  
    void setRank(int r) {  
        rank = r;  
    }  
  
    int getRank() {  
        return rank;  
    }  
};
```

a ideia é justamente impedir que a `main` mexa direto nos atributos privados. ela acessa e altera por meio dos getters e setters.

show
agora ex3

manda o ex 3 😎

tenta começar sozinho e me manda só onde travar.

agora o exercicio 3 fala de heranca. nunca fiz isso em
c++. como uma classe aluno herda de membroinatel?

em C++, a herança fica na declaração da classe com `:`.

Se `Aluno` herda de `MembroInatel`, fica assim:

```
</> C++  
  
class MembroInatel {  
public:  
    string nome;  
};
```

```
class Aluno : public MembroInatel {  
public:  
    int matricula;  
};
```

A parte importante é:

</> C++

```
class Aluno : public MembroInatel
```

Isso quer dizer: `Aluno` herda de `MembroInatel` .

Então um objeto `Aluno` também pode usar o que herdou da classe pai:

</> C++

```
Aluno a;
```

```
a.nome = "Caio";
```

```
a.matricula = 123;
```

Pensa assim:

`MembroInatel` = classe mais geral

`Aluno` = um tipo específico de `MembroInatel`

E esse `public` na herança, por enquanto, pode usar assim mesmo. É o mais comum nesses exercícios básicos.

seria `class aluno : public membroinatel?`

isso mesmo:

</> C++

```
class Aluno : public MembroInatel {  
    // coisas específicas de Aluno  
};
```

A ordem é sempre:

</> C++

```
class ClasseFilha : public ClassePai
```

Então Aluno herda tudo que for acessível de MembroInatel .

pq tem esse public depois dos dois pontos?

esse public depois dos : define como a herança vai acontecer.

```
</> C++
```

```
class Aluno : public MembroInatel
```

com public , o que era public em MembroInatel continua public dentro de Aluno .

bem resumido:

```
</> C++
```

```
class MembroInatel {  
public:  
    string nome;  
  
protected:  
    int idade;  
  
private:  
    int senha;  
};
```

se fizer:

```
</> C++
```

```
class Aluno : public MembroInatel
```

o Aluno consegue herdar/usar nome e idade conforme as regras de acesso, mas **não acessa diretamente** senha , porque ela é private .

pra sua atividade, pensa simplesmente assim:

```
: public MembroInatel = "Aluno herda publicamente de MembroInatel"
```

é o formato que você provavelmente vai usar no exercício.

como aluno consegue acessar nome que veio da classe pai?

se nome estiver public ou protected na classe pai, a classe Aluno consegue usar ele direto.

exemplo:

</> C++

```
class MembroInatel {  
protected:  
    string nome;  
};  
  
class Aluno : public MembroInatel {  
public:  
    void mostrarNome() {  
        cout << nome;  
    }  
};
```

aqui Aluno acessa nome normalmente porque ele é protected .

se nome fosse private , Aluno não poderia acessar direto. aí teria que usar um getter da classe pai:

</> C++

```
class MembroInatel {  
private:  
    string nome;  
  
public:  
    string getNome() {  
        return nome;  
    }  
};
```

e na filha:

</> C++

```
cout << getNome();
```

pra herança, protected é bem comum justamente porque a classe filha pode acessar, mas a main não acessa direto.

precisa colocar `virtual` no exercício 3 ou como eu vou chamar direto `aluno.seapresentar` não precisa?

se você vai chamar direto assim:

```
</> C++
```

```
aluno.seApresentar();
```

não precisa de `virtual`.

`virtual` é mais importante quando você usa **polimorfismo**, tipo uma variável/ponteiro da classe pai apontando para um objeto da classe filha:

```
</> C++
```

```
MembroInatel *m = &aluno;  
m->seApresentar();
```

ai, se `seApresentar()` for `virtual` na classe pai, o C++ chama a versão de `Aluno`.

pro seu exercício, se é só herança e chamada direta no objeto `Aluno`, pode fazer sem `virtual`.

```
posso instanciar aluno a; e depois fazer a.nome = "caio";  
a.curso = "engenharia de software";?
```

pode, desde que `nome` e `curso` estejam acessíveis como `public`.

```
</> C++
```

```
Aluno a;  
  
a.nome = "caio";  
a.curso = "engenharia de software";
```

Se `nome` veio da classe `MembroInatel` e lá ele estiver `protected` ou `private`, aí a `main` não consegue fazer `a.nome = ...` direto.

Nesse caso você usaria `setter`:

```
</> C++
```

```
a.setNome("caio");
```



```
a.setCurso("engenharia de software");
```

Então a regra é: na `main` , acesso direto só se o atributo for `public` .

agora o exercicio 4 parece parecido com o 3 mas pede `virtual`. qual a diferenca?

a diferença é que agora provavelmente o exercício quer que você use **polimorfismo**.

Sem `virtual` , se você tiver algo assim:

```
</> C++
```

```
MembroInatel *m = &aluno;  
m->seApresentar();
```

o C++ tende a chamar a versão de `MembroInatel` .

Com `virtual` na classe pai:

```
</> C++
```

```
class MembroInatel {  
public:  
    virtual void seApresentar() {  
        cout << "sou membro do inatel";  
    }  
};
```

e na filha:

```
</> C++
```

```
class Aluno : public MembroInatel {  
public:  
    void seApresentar() {  
        cout << "sou aluno";  
    }  
};
```

aí:

```
</> C++
```

```
Aluno aluno;  
MembroInatel *m = &aluno;  
  
m->seApresentar();
```

vai chamar:

```
</> C++  
  
sou aluno
```

porque `virtual` faz o C++ olhar pro **tipo real do objeto**, e não só pro tipo do ponteiro.

No exercício 3 você chamava direto:

```
</> C++  
  
aluno.seApresentar();
```

então não precisava. No 4, se o enunciado pede `virtual`, quase certeza que quer testar justamente isso: classe pai + classe filha + método sobrescrito + ponteiro/referência da classe pai.

o que exatamente o `virtual` faz em c++? explica simples

`virtual` faz o C++ escolher a versão do método de acordo com o **objeto real**, mesmo quando você está acessando ele por um ponteiro ou referência da classe pai.

exemplo:

```
</> C++  
  
class Animal {  
public:  
    virtual void falar() {  
        cout << "animal";  
    }  
};  
  
class Cachorro : public Animal {  
public:  
    void falar() {  
        cout << "cachorro";  
    }  
};
```

agora:

```
</> C++
```

```
Cachorro c;  
Animal *a = &c;  
  
a->falar();
```

com `virtual`, imprime:

```
</> C++
```

```
cachorro
```

sem `virtual`, chamaria a versão da classe `Animal`.

então pensa assim:

virtual = "se uma classe filha tiver sua própria versão desse método, use a versão dela."

eu tentei `vector<hobbit> hobbits;` e colocar `jardineiro` nele. compilou/nao funcionou como eu esperava. pq?

porque `vector<hobbit>` guarda objetos do tipo `hobbit` **por valor**.

se `jardineiro` herda de `hobbit`, até pode acontecer de você conseguir inserir um `jardineiro`, mas a parte específica de `jardineiro` é "cortada". isso chama **object slicing**.

exemplo:

```
</> C++
```

```
class hobbit {  
public:  
    virtual void falar() {  
        cout << "hobbit";  
    }  
};  
  
class jardineiro : public hobbit {  
public:  
    void falar() {  
        cout << "jardineiro";  
    }  
};
```

se fizer:

```
</> C++  
  
vector<hobbit> hobbits;  
  
jardineiro j;  
hobbits.push_back(j);
```

o vetor guarda só a parte `hobbit` de `j`.

por isso, pra usar polimorfismo de verdade, normalmente você guarda ponteiros:

```
</> C++  
  
vector<hobbit*> hobbits;  
  
jardineiro j;  
hobbits.push_back(&j);
```

e depois:

```
</> C++  
  
hobbits[0]->falar();
```

com `falar()` sendo `virtual`, ele chama a versão de `jardineiro`.

resumindo: `vector<hobbit>` guarda cópias de `hobbits`; `vector<hobbit*>` consegue apontar também para objetos das classes filhas sem perder a parte específica delas.

meu código está chamando sempre fazeratividade da classe `hobbit` mesmo tendo `jardineiro` e `cozinheiro`. o que eu esqueci?

provavelmente faltou colocar `virtual` no método da classe pai.

Na classe `hobbit`:

```
</> C++  
  
class hobbit {  
public:  
    virtual void fazeratividade() {  
        cout << "atividade de hobbit";  
    }  
};
```

```
}  
};
```

E nas filhas:

</> C++

```
class jardineiro : public hobbit {  
public:  
    void fazeratividade() override {  
        cout << "cuidando do jardim";  
    }  
};  
  
class cozinheiro : public hobbit {  
public:  
    void fazeratividade() override {  
        cout << "cozinhando";  
    }  
};
```

Aí, se você tiver:

</> C++

```
vector<hobbit*> hobbits;
```

e colocar objetos `jardineiro` e `cozinheiro` nele, cada um chama sua própria versão.

Se estiver usando `vector<hobbit>` em vez de `vector<hobbit*>`, aí também dá problema por causa daquele **object slicing** que vimos antes.

ah entao se eu esquecer `virtual` ele chama o metodo da classe base?

isso, nesse caso de polimorfismo por ponteiro/referência da classe base.

Exemplo:

</> C++

```
hobbit* h = &jardineiro;  
h->fazeratividade();
```

- sem `virtual` → chama `hobbit::fazeratividade()`

- com `virtual` → chama `jardineiro::fazeratividade()`

Então pode guardar assim na cabeça:

sem `virtual` , o C++ olha pro tipo do ponteiro.

com `virtual` , ele olha pro tipo real do objeto.

preciso usar `override` nas classes filhas ou não é obrigatório?

não é obrigatório, mas é bem recomendado.

isso funciona:

</> C++

```
class jardineiro : public hobbit {  
public:  
    void fazeratividade() {  
        cout << "jardinagem";  
    }  
};
```

mas melhor é:

</> C++

```
class jardineiro : public hobbit {  
public:  
    void fazeratividade() override {  
        cout << "jardinagem";  
    }  
};
```

o `override` serve como uma checagem pro compilador: ele confirma que esse método realmente está sobrescrevendo um método `virtual` da classe pai.

se você errar o nome ou os parâmetros, ele te avisa. Então: `virtual` na classe pai, `override` nas filhas.